Workflow

Technical Blueprint for a Compliant, Explainable AI-Driven Digital Lending Platform in India

1. Introduction: The New Paradigm of Digital Lending

The landscape of credit risk assessment in India has undergone a seismic shift, driven by the convergence of ubiquitous digital payments, stringent regulatory oversight, and the maturation of artificial intelligence. The era of unregulated digital lending, characterized by opaque algorithms, coercive recovery practices, and data sovereignty violations, has been decisively ended by the Reserve Bank of India's (RBI) Digital Lending Guidelines (DLG) of 2023. For architects, data scientists, and product leaders, this necessitates a fundamental rethinking of how lending platforms are conceived, engineered, and operated.

Building a credit risk project today is no longer merely a data science challenge; it is a complex orchestration of legal compliance, ethical AI, and robust software engineering. The modern lending platform must solve the "thin-file" problem—assessing the creditworthiness of the hundreds of millions of Indians who transact digitally via the Unified Payments Interface (UPI) but lack a formal credit history (CIBIL score)—while simultaneously adhering to rigid standards of explainability and privacy. The traditional "black box" model, which outputs a score without justification, is functionally obsolete and legally perilous in a regime that demands transparency for every adverse action.

This report provides an exhaustive, step-by-step technical guide to constructing such a platform. It moves beyond high-level abstractions to detail the specific architectural patterns, feature engineering techniques, and algorithmic strategies required to build a system that is not only predictive but also compliant, fair, and transparent. We will explore the integration of Gradient Boosted Decision Trees (GBDTs) with monotonic constraints for regulatory safety, the deployment of SHAP (SHapley Additive exPlanations) for generating adverse action codes, and the use of DiCE (Diverse Counterfactual Explanations) to provide customers with actionable paths to approval. Furthermore, we will detail the full-stack engineering required to deliver this intelligence via an offline-first Progressive Web App (PWA) tailored for India's variable network conditions.

2. Regulatory Compliance as Architecture

In the current ecosystem, compliance cannot be an afterthought; it must be the foundation of the system architecture. The distinction between Regulated Entities (REs)—banks and NBFCs—and Lending Service Providers (LSPs)—fintech platforms—dictates the flow of funds, data, and liability.

2.1 The Direct Fund Flow Mechanism

The most critical architectural constraint imposed by the RBI is the prohibition of "pass-through" accounts. Historically, fintechs would pool borrower repayments into their own nodal accounts before settling with the lender. This practice is now explicitly banned. The architecture must

enforce a direct flow of funds between the borrower and the RE.

2.1.1 Engineering the Bypass Architecture

To implement this, the backend microservices cannot process funds directly. Instead, the payment gateway integration must be configured in a "Nodal Bypass" or "Direct Settlement" mode.

- **Disbursement:** When a loan is approved, the Loan Origination System (LOS) sends a signal to the RE's Core Banking System (CBS) or a connected payment switch. The transfer is initiated directly from the RE's regulated bank account to the borrower's validated bank account. The fintech platform receives only a status webhook (Success/Failure/Pending).
- **Repayment:** Similarly, repayment mandates (e-NACH or UPI Autopay) must be registered in favor of the RE. The fintech app acts solely as a UI layer for initiating the transaction, but the beneficiary VPA (Virtual Payment Address) or account number must belong to the lender.

Implications for Database Design: The transaction ledger in the fintech's database must be strictly a "Shadow Ledger." It records the *intent* and *confirmation* of transactions but does not serve as the legal record of funds. Reconciliation services must be built to constantly sync this shadow ledger with the RE's bank statements to detect discrepancies, but the system must treat the RE's record as the single source of truth.

2.2 Data Sovereignty and the DPDP Act

The Digital Personal Data Protection (DPDP) Act 2023 introduces stringent requirements for Data Fiduciaries. The architecture must enable granular consent and absolute data sovereignty.

2.2.1 Data Localization Strategy

All customer data must reside on servers physically located within India. This requirement extends beyond the primary database to include backups, logs, and analytics warehouses.

- **Cloud Selection:** If using hyperscalers like AWS or Google Cloud, resources must be strictly pinned to the ap-south-1 (Mumbai) or ap-south-2 (Hyderabad) regions.
- **Disaster Recovery (DR):** Cross-region replication for DR purposes must be contained within Indian borders. Replicating data to a Singapore or Frankfurt region for redundancy is a violation of the DLG.

2.2.2 The "Purpose Limitation" Architecture

The system must be designed to purge data once its specific purpose is fulfilled. This is a radical departure from the "store everything forever" philosophy of big data.

- **Retention Policies:** The system must distinguish between "KYC Data" (which must be retained for statutory periods, typically 5-10 years under PMLA rules) and "Alternative Data" (e.g., SMS logs used for underwriting). Once the loan decision is made, if the user is rejected, the granular SMS logs should ideally be purged or anonymized, retaining only the derived score, unless specific consent is maintained for future assessments.
- **Revocation Workflows:** The backend must expose an API endpoint for consent revocation. Triggering this endpoint must initiate a cascade deletion process across all

microservices—removing the user's data from the marketing service, the underwriting cache, and the analytics data lake within a legally defined timeframe.

2.3 Key Fact Statement (KFS) Engine

The guidelines mandate that a Key Fact Statement (KFS) containing the all-inclusive Annual Percentage Rate (APR) be presented to the borrower *before* the contract is signed. This creates a latency constraint: the pricing engine must calculate the final risk-based interest rate, add all processing fees, verification charges, and third-party costs, and compute the APR in real-time.

Table 1: Components of the KFS Calculation Engine

Component	Description	Technical Implication
Base Rate	The lender's cost of funds + margin.	Retrieved from the RE's product configuration service.
Risk Premium	Add-on rate based on the ML model's score.	Dynamic output from the Inference Engine.
Processing Fee	Operational cost charged to the user.	Must be explicitly added to the APR formula.
Third-Party Charges	Costs for credit bureau pulls or KYC verification.	If passed to the user, these must be included in the APR.
APR Formula	Regulatory formula for annualized cost.	\frac{\text{Total Cost}}{\text{Loan Amount}} \times \frac{365}{\text{Tenure}} \times 100

The KFS is not just a UI element; it is a legally binding data structure. The system must store a snapshot of the exact KFS presented to the user, stamped with a cryptographic hash to prove non-repudiation in case of future disputes.

3. Data Strategy for Thin-File Borrowers

In the absence of robust credit bureau data (CIBIL/Experian) for "New-to-Credit" (NTC) populations, the platform must construct a "synthetic" credit profile using alternative data. This requires a sophisticated data ingestion and feature engineering pipeline that transforms behavioral footprints into credit signals.

3.1 The Hierarchy of Alternative Data

We can categorize alternative data based on its predictive power and accessibility. The architecture must prioritize sources that offer the highest signal-to-noise ratio while remaining compliant with privacy norms.

3.1.1 Unified Payments Interface (UPI) Transaction Logs

UPI data is the gold standard for income estimation in the informal economy. Unlike bank statements which can be infrequent, UPI logs capture high-velocity, low-value transactions that reveal daily cash flow patterns.

- **Income Stability Analysis:** The system must parse narratives to identify recurring inflows. In the gig economy, "salary" is rarely labeled as such. Instead, the system looks

for "Cluster Consistency"—recurring payments from the same VPA (e.g., a Zomato or Swiggy payout wallet) arriving at similar intervals.

- **Network Centrality:** By analyzing the P2P (Person-to-Person) graph, the system can calculate the user's "Eigenvector Centrality." A user who transacts with other highly-rated users or established merchants is statistically less likely to default than a user who transacts in a closed loop with known defaulters.

3.1.2 SMS Parsing and Financial Hygiene

While the RBI prohibits accessing personal messages, reading transactional SMS (with explicit consent) remains a primary method for scraping financial data on Android devices in India.

- **Regex Engines:** The system utilizes complex Regular Expressions (Regex) to parse standardized SMS formats from banks (e.g., "Acct XX123 debited for Rs 500").
- **Feature Extraction:** The goal is to construct a "Virtual Bank Statement." Key features include the "Average Daily Balance" and the "Ratio of Insufficient Funds (NSF) Alerts." A user who frequently receives "Low Balance" warnings via SMS presents a high liquidity risk, regardless of their total income.

3.1.3 Utility and Bill Payment Data

Data from the Bharat Bill Payment System (BBPS) acts as a proxy for "Willingness to Pay." Regular payment of electricity, water, and mobile postpaid bills indicates organizational discipline.

- **Latency Proxy:** The system calculates the gap between the "Bill Generation Date" and the "Bill Payment Date." A user who consistently pays on the due date (Gap = 0) is reliable; a user who pays 3 days late (Gap = -3) shows signs of stress or disorganization.

3.2 Feature Engineering for the Informal Sector

The raw data must be transformed into mathematical features that a GBDT model can ingest. For the Indian informal sector, standard features like "Debt-to-Income Ratio" are often calculable only through proxies.

3.2.1 Volatility and Stability Indices

Income volatility is the enemy of repayment. A gig worker earning ₹50,000 one month and ₹5,000 the next is higher risk than a salaried worker earning a steady ₹20,000.

- **Coefficient of Variation (CV):** We calculate the CV of monthly inflows over a 6-month window (CV = \frac{\sigma}{\mu}). A lower CV indicates stable income.
- **Sawtooth Analysis:** We analyze the daily balance curve. A "Sawtooth" pattern—where funds arrive and are immediately withdrawn to near zero—indicates a lack of savings buffer. The "Time to Zero" (average days from income receipt to zero balance) is a critical predictive feature.

3.2.2 The Affordability Index

Instead of a simple Debt-to-Income ratio, we construct an "Affordability Index."

- **Disposable Income Calculation:** \text{Estimated Income} - (\text{Rent Proxy} +

\text{Utility Avg} + \text{Existing EMI Proxy}).

- **Rent Proxy:** Identified by large, recurring monthly debits to individual accounts.
- **Existing EMI Proxy:** Identified by keywords like "Bajaj", "Muthoot", "HDB" in transaction logs.
- **Logic:** If the requested EMI > 40% of the calculated Disposable Income, the system flags the application as high risk, regardless of the user's raw income.

4. Machine Learning Model Development

The core decision engine utilizes Gradient Boosted Decision Trees (GBDTs). While Deep Learning (Neural Networks) excels at unstructured data (images, text), GBDTs like XGBoost, LightGBM, and CatBoost remain superior for tabular financial data due to their handling of discrete features, missing values, and interpretability.

4.1 Algorithm Selection: LightGBM

LightGBM is often preferred in production environments over XGBoost for two reasons: speed (leaf-wise growth) and efficient handling of large datasets.

- **Handling Missing Data:** Thin-file borrowers will inherently have missing features (e.g., no utility bill history). LightGBM handles NaN values natively by learning the optimal direction (left or right split) for missing entries during training, which is far superior to arbitrary mean/median imputation which distorts the data distribution.

4.2 Monotonic Constraints for Regulatory Safety

A critical, often overlooked requirement in credit modeling is **Monotonicity**. In a regulated environment, the relationship between specific risk factors and the output score must be directionally consistent. It is legally indefensible to reject a user *because* their income increased, yet complex non-linear models can sometimes learn such spurious correlations due to noise in the training data.

Implementation Strategy: We explicitly enforce monotonic constraints during model training.

- **Income Features:** Constraint = +1 (As Income increases, Score/Probability of Approval must increase).
- **Delinquency Features:** Constraint = -1 (As number of past defaults increases, Score must decrease).
- **Utilization Features:** Constraint = -1 (As credit utilization increases, Score decreases).

Code Structure for Constraints: In the LightGBM configuration, we pass a vector of constraints:

```
# 1 = Increasing, -1 = Decreasing, 0 = No Constraint
monotone_constraints = [1, -1, -1, 0, 1,...]
model = lgb.LGBMClassifier(monotone_constraints=monotone_constraints)
```

This ensures that the model's decision surface creates a "fair" and justifiable policy, preventing "black box" anomalies where better financial behavior leads to worse outcomes.

4.3 Probability Calibration

The raw output of a GBDT classifier is not a probability; it is a log-odds score (logit). To perform risk-based pricing (setting the APR based on risk), we need an accurate probability of default (PD).

- **Calibration Technique:** We utilize **Isotonic Regression**. Unlike Platt Scaling (Sigmoid), which assumes a specific S-shaped distribution, Isotonic Regression fits a non-parametric stepwise function. This is ideal for credit scoring where the distribution of risk scores is often skewed.
- **Evaluation Metric:** The model is optimized not just for AUC-ROC (ranking quality) but for **Brier Score** (calibration quality). A well-calibrated model ensures that among 100 applicants predicted to have a 10% default risk, close to 10 actually default. This accuracy is vital for financial forecasting and provisioning.

5. Explainable AI (XAI) and Adverse Action Generation

The RBI guidelines and global best practices (like the Equal Credit Opportunity Act in the US) mandate that lenders provide specific reasons for denial. This requires unwrapping the "Black Box" of the ML model.

5.1 SHAP (SHapley Additive exPlanations)

We employ SHAP, a method rooted in cooperative game theory, to attribute the model's final score to individual feature contributions.

- **The Concept:** For a specific prediction, SHAP calculates the marginal contribution of each feature value towards the output, compared to a baseline.
- **TreeExplainer:** We use shap.TreeExplainer, which is an optimized, exact algorithm for tree-based models (like LightGBM). This is significantly faster than the model-agnostic KernelSHAP, making it feasible for real-time inference.

5.2 The Reason Code Generation Pipeline

Raw SHAP values (e.g., income_feature: -0.45) are unintelligible to consumers. We must convert these into "Adverse Action Codes" or "Reason Codes."

Step-by-Step Algorithm:

1. **Compute SHAP Values:** For the rejected applicant, generate the vector of SHAP values.
2. **Filter Negative Contributors:** Isolate features that pushed the score *down* (towards rejection).
3. **Rank by Magnitude:** Sort these negative features by their absolute SHAP value (impact).
4. **Semantic Mapping:** Map the technical feature names to regulatory-approved explanation strings using a lookup table.
    - utilization_30d \rightarrow "Ratio of balance to credit limit is too high."
    - inq_6m \rightarrow "Too many recent applications for credit."
    - upi_volatility \rightarrow "Income sources appear irregular."
5. **Selection:** Select the top 3-4 reasons to display in the rejection letter/email.

Table 2: Mapping Technical Features to Consumer Reasons

Technical Feature	SHAP Value	Adverse Action Reason Code
avg_bal_below_min	-0.85	"Average monthly balance is consistently low."

Technical Feature	SHAP Value	Adverse Action Reason Code
recent_bounced_checks	-0.62	"History of payment failures observed."
credit_utilization_ratio	-0.41	"Current debt usage is high relative to limits."

This automated pipeline ensures that every rejection is accompanied by a transparent, mathematically derived explanation, satisfying the "Right to Explanation" requirements.

6. Customer Recourse: Counterfactual Analysis

While SHAP explains *why* a rejection happened, it does not tell the user *how* to fix it. To empower the customer (and improve future conversion), we implement Counterfactual Explanations.

6.1 DiCE (Diverse Counterfactual Explanations)

We integrate the Microsoft DiCE library to generate actionable "What-If" scenarios.

- **The Optimization Problem:** DiCE solves an optimization problem to find a perturbed feature vector x' such that the model predicts "Approve" for x', while minimizing the distance between x' and the original input x.
- **Diversity:** DiCE generates multiple diverse paths. For example, Path A might involve increasing income, while Path B involves reducing debt. This gives the user agency to choose the path that suits their situation.

6.2 Implementation Constraints for Realism

Generating counterfactuals blindly can lead to absurd advice (e.g., "Change your age to 45" or "Reduce your number of children").

- **Immutable Features:** We explicitly configure DiCE with a list of immutable features (Age, Gender, ZipCode, History_of_Defaults). The algorithm is constrained to only vary mutable features like Current_Balance, Utilization, or Recent_Inquiries.
- **Feasibility Constraints:** We also impose range constraints. We typically constrain income increases to reasonable bounds (e.g., +20%) to avoid suggesting impossible salary jumps.
- **Actionable Advice:** The system translates the valid counterfactual into a user-friendly prompt: *"If you pay down ₹5,000 of your current credit card debt, your approval probability increases to 85%."*.

7. Full-Stack Engineering Architecture

The deployment architecture must support high concurrency, strictly segregate data for security, and function reliably on the unstable networks typical of Tier-3 Indian cities.

7.1 Microservices Design with FastAPI

The backend is built using Python's **FastAPI** framework. Python is chosen to minimize the "impedance mismatch" between the Data Science (ML) environment and the Production

Application environment.

- **Service Decomposition:**
  - **Gateway Service:** Handles auth, rate limiting, and request routing.
  - **LOS (Loan Origination Service):** Manages the state machine of the loan application (Draft -> Submitted -> Underwriting -> Approved/Rejected).
  - **Inference Service:** A dedicated, scalable container that hosts the LightGBM model, SHAP explainer, and DiCE engine. Isolating this is crucial because explainability computations are CPU-intensive and should not block the I/O-bound API traffic.
- **Asynchronous Processing:** FastAPI's async capabilities are leveraged to handle external API calls (e.g., fetching a CIBIL report or scraping SMS data) without blocking the main thread.

7.2 Database Schema and Data Lifecycle

We utilize **PostgreSQL** as the primary transactional store.

- **Schema Design for Compliance:**
  - users: Stores PII (Encrypted at rest).
  - loan_applications: Stores the state and KFS snapshot.
  - consent_ledger: A dedicated table tracking every permission granted (purpose, timestamp, revocation_status).
- **Data Partitioning:** To comply with retention policies, data is often partitioned by time. This makes it efficient to drop old logs (e.g., drop partition sms_logs_2020) when the retention period expires, ensuring compliance with the DPDP Act's storage limitations.

7.3 Offline-First Frontend with Next.js (PWA)

The user interface is built as a **Progressive Web App (PWA)** using **Next.js**.

- **Service Workers for Offline Resilience:** We use service workers (next-pwa) to cache the "App Shell" (HTML/CSS/JS bundles). This ensures the app loads instantly, even on 2G networks or in airplane mode.
- **Background Sync:** If a user completes an application while offline, the data is serialized and stored in the browser's IndexedDB. The Service Worker listens for the sync event (when connectivity returns) and silently pushes the payload to the backend. This prevents data loss in areas with spotty internet.
- **UI Performance:** We utilize lightweight component libraries like **Mantine** or **Chakra UI**. These libraries are tree-shakable (only used code is bundled), minimizing the JavaScript payload size, which is critical for low-end Android devices common in the target demographic.

7.4 Security Architecture

- **Encryption:** All sensitive fields (PAN, Aadhaar) are encrypted at the application level before insertion into the database using AES-256.
- **API Security:** The internal ML service is not exposed to the internet. Communication between the Gateway and the ML Service happens over a private Docker network. External traffic is guarded by strict CORS policies and Rate Limiting (using Redis) to prevent scraping or DDoS attacks.

8. Operationalization and Monitoring

Deploying the model is merely the beginning. Financial behavior is dynamic, and models degrade over time.

8.1 Drift Detection and Retraining

The system must monitor for two types of drift:

- **Data Drift (Covariate Shift):** Has the demographic of applicants changed? (e.g., younger applicants entering the pool). We use the **Population Stability Index (PSI)** to monitor feature distributions. A PSI > 0.2 triggers an alert.
- **Concept Drift:** Has the relationship between features and risk changed? (e.g., high income becoming less protective due to inflation). We monitor the **Target Distribution** (default rates) in real-time.
- **Retraining Pipeline:** When drift is detected, an automated pipeline (Airflow/Kubeflow) is triggered to retrain the LightGBM model on the most recent window of data, validating the new model against the holdout set before promoting it to production.

8.2 Fairness Auditing

We implement a **Disparate Impact Analysis** module. This scheduled job (running weekly/monthly) compares the approval rates across protected classes (Gender, Location). If the approval rate for a protected group falls below 80% of the reference group (the "Four-Fifths Rule"), the system flags the model for a bias audit, prompting data scientists to investigate potential algorithmic bias.

9. Conclusion

Building a digital lending platform for the Indian market requires a synthesis of advanced technology and deep regulatory awareness. By leveraging **UPI-based feature engineering**, the system solves the access problem for thin-file borrowers. By integrating **monotonic GBDTs**, **SHAP**, and **DiCE**, it solves the transparency problem, transforming the "black box" into an explainable, compliant decision engine. Finally, by adopting an **offline-first PWA architecture**, it ensures that this sophisticated financial tool is accessible to the users who need it most, regardless of their device or network quality. This blueprint represents not just a technical specification, but a roadmap for responsible, inclusive financial innovation in the post-DLG era.

10. Detailed Implementation Guide

Step 1: Data Pipeline Setup & Feature Engineering

Objective: Transform raw transactional logs into a feature vector suitable for machine learning, focusing on "thin-file" proxies.
Technical Implementation: We assume access to a transaction dataset (e.g., a CSV export of bank statements or parsed SMS logs).

1. **Data Ingestion & Cleaning:**

- Load data using Pandas.
- Convert timestamp to datetime objects.
- Fill NaN narratives with "UNKNOWN".

2. **Categorization Logic:** Implement keyword-based tagging to classify transactions.
```
def tag_transaction(narrative):
    narrative = narrative.lower()
    if any(x in narrative for x in ['zomato', 'swiggy',
'mcdonalds']): return 'dining'
    if any(x in narrative for x in ['bescom', 'water',
'recharge']): return 'utility'
    if any(x in narrative for x in ['salary', 'credit',
'payout']): return 'income'
    return 'other'
```

3. **Advanced Feature Construction (The "Thin-File" Proxies):**
   - **Income Proxy:** Identify the largest recurring monthly credit or the sum of P2P inflows from consistent senders.
   - **Stability Index (CV):**
     ```
     daily_balances = df.groupby('date')['balance'].last()
     stability_index = daily_balances.std() /
     daily_balances.mean()
     # Lower is better (more stable)
     ```
   - **Cash Flow Ratio:** \frac{\sum \text{Credits}}{\sum \text{Debits}}. A ratio < 1 suggests the user is spending down savings.
   - **Digital Intensity:** Count of unique merchant VPAs. High diversity correlates with higher creditworthiness.
4. **Synthetic Data Generation:** Use Faker to generate synthetic transaction histories for testing. Simulate profiles like "Student" (low income, high P2P) vs. "Salaried" (fixed income, high utility payments) to validate feature logic.

Step 2: Model Training with Monotonic Constraints

Objective: Train a robust, compliant LightGBM model.
Technical Implementation:

1. **Monotonicity Configuration:** Define constraints to ensure logical behavior.
```
# Dictionary mapping feature names to constraints (1: increasing,
-1: decreasing)
constraints = {
    "estimated_income": 1,      # Higher income -> Higher score
    "stability_index": -1,      # Higher instability -> Lower
score
    "recent_bounces": -1,       # More bounces -> Lower score
    "age": 0                    # No constraint
}
```

2. **Training:** Train the LGBMClassifier using these constraints.
```
model =
```

```
lgb.LGBMClassifier(monotone_constraints=constraints_vector)
model.fit(X_train, y_train)
```

3. **Calibration:** Apply Isotonic Regression to convert scores to probabilities.
```
from sklearn.calibration import CalibratedClassifierCV
calibrated_model = CalibratedClassifierCV(model,
method='isotonic', cv='prefit')
calibrated_model.fit(X_val, y_val)
```

Step 3: Integrating Explainability (SHAP & Reason Codes)

Objective: Generate adverse action codes for denied applicants.
Technical Implementation:

1. **SHAP Calculation:**
```
import shap
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(applicant_data)
```

2. **Reason Code Mapping:** Create a function to extract top negative contributors.
```
def get_adverse_codes(shap_vals, feature_names, mapping_dict):
    # Filter for negative impact (pushing towards rejection)
    negative_impacts = [(feature_names[i], val) for i, val in
enumerate(shap_vals) if val < 0]
    # Sort by magnitude
    negative_impacts.sort(key=lambda x: abs(x), reverse=True)
    # Map to text
    reasons =]
    return reasons
```

Step 4: Building the Counterfactual Engine (DiCE)

Objective: Provide the user with a path to future approval.
Technical Implementation:

1. **Initialize DiCE:**
```
import dice_ml
d = dice_ml.Data(dataframe=train_df,
continuous_features=['income', 'balance'], outcome_name='target')
m = dice_ml.Model(model=model, backend='sklearn')
exp = dice_ml.Dice(d, m)
```

2. **Generate with Constraints:**
```
query_instance = applicant_data
# Immutable features: Cannot change Age or Past Defaults
dice_exp = exp.generate_counterfactuals(
    query_instance,
    total_CFs=3,
```

```
    desired_class="opposite",
    features_to_vary=['balance', 'utilization'], # Only mutable
features
    permitted_range={'income': [current_income, current_income *
1.2]} # Max 20% increase
)
```

Step 5: Backend & Frontend Integration

Objective: Deploy the system via Microservices and PWA.

1. **FastAPI Backend:**
    - Create endpoints for /predict (Model), /explain (SHAP), and /recourse (DiCE).
    - Use Pydantic models to validate input JSON schemas.
    - Load the model into memory only once at startup (@app.on_event("startup")).
2. **Next.js PWA:**
    - Configure next-pwa in next.config.js.
    - Create manifest.json for "Add to Home Screen".
    - Implement ServiceWorker to cache the dashboard shell.
    - Use IndexedDB to store loan application drafts locally if the user goes offline.
3. **Deployment:**
    - Dockerize the application.
    - Use Docker Compose to spin up the API, Database, and Redis Cache in an isolated network.
    - Deploy to an India-based cloud region (e.g., AWS Mumbai) to satisfy data localization laws.

Works cited

1. Home Credit Default Risk | Kaggle, https://www.kaggle.com/c/home-credit-default-risk 2. The organisation of digital payments in India - lessons from the Unified Payments Interface (UPI) - Bank for International Settlements, https://www.bis.org/publ/bppdf/bispap152_e_rh.pdf 3. RBI's Digital Lending Guidelines: A Legal Deep-Dive for India's Fintech Ecosystem, https://theattorneys.co/rbis-digital-lending-guidelines-a-legal-deep-dive-for-indias-fintech-ecosystem/ 4. RBI on AI Risks: Takeaways for Users - BillCut, https://www.billcut.com/blogs/rbi-on-ai-risks-takeaways-for-users/ 5. Digital Lending Guidelines - Reserve Bank of India, https://www.rbi.org.in/commonman/english/scripts/FAQs.aspx?Id=3413 6. Google Cloud RBI Digital Lending Guidelines Whitepaper, https://cloud.google.com/security/compliance/rbi_guidelines_digital_lending_googlecloud_whitepaper 7. Impact of the Digital Personal Data Protection Act 2023 - JISA Softech, https://www.jisasoftech.com/impact-of-the-digital-personal-data-protection-act-2023-on-businesses-in-india/ 8. RBI-GUIDELINES-ON-DIGITAL-LENDING-02-09-22.pdf - FIDC India, https://fidcindia.org.in/wp-content/uploads/2022/09/RBI-GUIDELINES-ON-DIGITAL-LENDING-02-09-22.pdf 9. The Impact of India's New Digital Personal Data Protection Rules - Privacy World, https://www.privacyworld.blog/2025/04/the-impact-of-indias-new-digital-personal-data-protection-rules/ 10. India's DPDPA 2023 and Its Impact on the Financial Sector - eMudhra, https://emudhra.com/en/blog/implications-of-indias-dpdpa-2023-on-the-financial-sector 11. The

Role of Nonbanks and Fintechs in Boosting India's UPI Person-to-Merchant Transactions - Federal Reserve Bank of Kansas City, https://www.kansascityfed.org/research/payments-system-research-briefings/the-role-of-nonbanks-and-fintechs-in-boosting-indias-upi-person-to-merchant-transactions/ 12. Case Study: India World Bank Fast Payments Toolkit, https://fastpayments.worldbank.org/sites/default/files/2021-10/World_Bank_FPS_India_IMPS_and_UPI_Case_Study.pdf 13. Credit Risk Feature Engineering with Python & Numba | by Muanai Khalifah Revindo, https://medium.com/@muanaikhalifahr/credit-risk-feature-engineering-with-python-numba-eb9643908f9c 14. New credit-risk models for the unbanked - McKinsey & Company, https://www.mckinsey.com/capabilities/risk-and-resilience/our-insights/new-credit-risk-models-for-the-unbanked 15. Feature Engineering Techniques for Quantitative Models – Blog - BlueChip Algos, https://bluechipalgos.com/blog/feature-engineering-techniques-for-quantitative-models/ 16. The Importance of Income Validation in Lending: Safer Loans, Stronger Compliance, https://www.proteantech.in/articles/income-validation-02-06-2025/ 17. Parameters — LightGBM 4.6.0.99 documentation, https://lightgbm.readthedocs.io/en/latest/Parameters.html 18. monotonic, http://ethen8181.github.io/machine-learning/trees/monotonic.html 19. Application of Monotonic Constraints in Machine Learning Models | by Ajay Tiwari - Medium, https://medium.com/analytics-vidhya/application-of-monotonic-constraints-in-machine-learning-models-334564bea616 20. 1.16. Probability calibration — scikit-learn 1.8.0 documentation, https://scikit-learn.org/stable/modules/calibration.html 21. Probability Calibration. In this tutorial I will talk about how… | by Tola Alade | Cambridge Spark, https://blog.cambridgespark.com/probability-calibration-c7252ac123f 22. shap/shap: A game theoretic approach to explain the output of any machine learning model. - GitHub, https://github.com/shap/shap 23. The credit risk explainability and transparency problem: Adverse Action Codes assignment for W.o.E. Logistic Regression Scorecards and advanced Machine Learning Algorithms | by Levorato Giampaolo | Oct, 2025 | Medium, https://medium.com/@levorato.giampaolo/the-credit-risk-explainability-and-transparency-problem-adverse-actions-codes-assignment-for-w-o-e-ffc9d524e796 24. Create reason code via SHAP value for GMB model | by Wen Zhang - Medium, https://medium.com/@wenzhang.ds/create-reason-code-via-shap-value-for-gmb-model-2e150837c8fe 25. Using Explainable AI to Produce ECOA Adverse Action Reasons: What Are The Risks?, https://www.paceanalyticsllc.com/post/ecoa-adverse-actions-and-explainable-ai 26. interpretml/DiCE: Generate Diverse Counterfactual Explanations for any machine learning model. - GitHub, https://github.com/interpretml/DiCE 27. Diverse Counterfactual Explanations (DiCE) for ML — DiCE 0.12 documentation, https://interpret.ml/DiCE/ 28. 15 Counterfactual Explanations – Interpretable Machine Learning - Christoph Molnar, https://christophm.github.io/interpretable-ml-book/counterfactual.html 29. Counterfactual Explanation of AI Models Using an Adaptive Genetic Algorithm With Embedded Feature Weights - IEEE Xplore, https://ieeexplore.ieee.org/iel7/6287639/10380310/10536083.pdf 30. How to Build an Instant Machine Learning Web Application with Streamlit and FastAPI, https://developer.nvidia.com/blog/how-to-build-an-instant-machine-learning-web-application-with-streamlit-and-fastapi/ 31. Serving ML Models as Microservices with FastAPI, Docker, and Docker-Compose - Tobias Larysch, https://tlary.github.io/post/fastapi/ 32. Nneji123/fastapi-nextjs: This project demonstrates the integration of FastAPI, SQLModel, PostgreSQL, Redis, Next.js, Docker, and Docker Compose, showcasing essential API features like rate limiting, pagination, logging, and metrics using Prometheus and Grafana. - GitHub, https://github.com/Nneji123/fastapi-nextjs 33. Guides: PWAs | Next.js,

https://nextjs.org/docs/app/guides/progressive-web-apps 34. How to make a Next.js app a PWA with offline support | Progressive Web App - YouTube, https://www.youtube.com/watch?v=MHjF166fcxw 35. Building Native-Like Offline Experience in Next.js PWAs | Fishtank, https://www.getfishtank.com/insights/building-native-like-offline-experience-in-nextjs-pwas 36. Top 10 Next.js UI Libraries in 2025 | RetroUI Blogs, https://www.retroui.dev/blogs/top-10-nextjs-ui-library 37. Best UI Libraries to Use in 2025 - Aubergine Solutions, https://www.aubergine.co/insights/top-ui-libraries-to-use-in-2025 38. Creating a Secure Machine Learning API with FastAPI and Docker, https://machinelearningmastery.com/creating-a-secure-machine-learning-api-with-fastapi-and-docker/ 39. RBI Committee Report on Responsible AI in the Financial Sector (FREE-AI Framework), https://chambers.com/articles/rbi-committee-report-on-responsible-ai-in-the-financial-sector-free-ai-framework 40. Generating Synthetic Bank Transactional Data Using Markov Models - Medium, https://medium.com/@samvardhan777/generating-synthetic-bank-transactional-data-using-markov-models-bde50be23886

Ideation

Public-Facing Explainable Credit Risk Engine (India) – Project Overview

Objective: Build a web-based prototype credit-scoring service for Indian lenders that ingests borrower data from users (via forms or uploads), scores credit risk with machine learning, and provides transparent explanations for each decision. This PoC uses public Kaggle datasets (e.g. Home Credit Default Risk, bank transaction data) to train models like gradient-boosted trees and small GRU/LSTM networks. The system is containerized (Docker) with a Next.js frontend and a RESTful backend API. Crucially, every decision is accompanied by an explanation (SHAP feature contributions, “reason codes,” and counterfactual suggestions) to meet India’s regulatory emphasis on transparency and fairness.

Architecture & Key Components

The system comprises the following layers and services:

Frontend (Next.js Web UI): A responsive, mobile-first interface collects borrower inputs (e.g. personal/profile data, income, liabilities) and displays results. The UI adapts to Indian conditions – it supports Hindi/English, simple layouts and even feature-phone/USSD-style fallbacks. After scoring, it shows the approval/denial outcome, detailed factor contributions, and “reason codes.” It also clearly discloses loan terms (interest rates, fees) in compliance with RBI transparency rules.

Backend API (Python/Flask or FastAPI, Dockerized): Exposes endpoints for data submission, scoring, and explanation retrieval. All logic (data validation, feature lookup, model inference) runs here. The API is containerized for portability and can be orchestrated with tools like Docker Compose or Kubernetes. Sensitive data is handled per Indian privacy laws (e.g. DPDP 2023) – we avoid storing unnecessary personal data and encrypt any logs.

Orchestrator / MLOps Pipeline: A workflow engine (e.g. Apache Airflow, Prefect or Kubeflow) automates data training pipelines. On a schedule or triggered by new data, it reads cleaned Kaggle datasets and any aggregated user data, performs feature processing, retrains models, calibrates probabilities, and publishes new model versions. Logging ensures auditability (model versions, data used) for governance.

Data & Feature Store: Raw input and intermediate features are stored in a database (e.g. PostgreSQL or NoSQL). A feature store (e.g. Feast or a custom service) serves consistent features to training and serving. The store holds engineered attributes (see below), so that the same values (and pre-processing logic) are applied at inference time. Historical raw data is retained (with PII stripped) for offline analysis and model retraining.

Model Stack: The core ML models may include tree-based ensembles (LightGBM/CatBoost) for tabular data and a small GRU/LSTM network to capture any sequential behavior (e.g. monthly transaction or payment history). We enforce monotonicity constraints on key features where appropriate (to aid interpretability) – for example, higher income should not reduce score. All models output a default probability that is then calibrated (e.g. via isotonic regression) to align with true risk.

Explainability Module: A dedicated component computes explanations for each prediction. We

use model-agnostic tools like SHAP to break down a score into feature contributions. For each applicant, the module identifies the top contributing factors and generates simple “reason codes” (e.g. “High existing debt,” “Low income,” “Very new credit file”). We also generate minimal-change counterfactuals: for a denial, we compute the smallest adjustment (e.g. income +20%) that would flip the decision. These explanations are returned via the API and shown on the frontend in human-friendly language.

Storage & Audit: All decisions and inputs are logged for audit. This includes storing (anonymized) applicant profiles, model version, score, and explanation vectors. This audit trail aligns with RBI’s guidance that AI decisions be auditable and transparent. Data retention respects DPDP and RBI IT Guidelines (no excessive collection).

Data Ingestion & Feature Engineering

Training Data: We leverage publicly available datasets for initial model building. For example, the Kaggle Home Credit Default Risk dataset includes borrower demographics, credit history, and past application records. We also use bank transaction datasets to simulate spending/income patterns. These provide features like credit bureau scores, past delinquency flags, and aggregate credit usage.

User Input Ingestion: Borrowers (or lenders on their behalf) can submit new data via an online form or CSV upload. The frontend sends this to the API, which validates types and ranges. We mimic typical loan-app data: age, employment type, income, past defaults, etc. Missing or noisy fields are handled by imputation (mean/median or predictive imputation) as in our Kaggle training pipeline.

Feature Engineering: We construct derived features to capture risk signals. This includes common financial ratios (e.g. debt-to-income, credit utilization) and time-series summaries (e.g. number of months since last delinquency). Categorical variables (occupation, education) are encoded (one-hot or embeddings). We also apply transformations (binning or Box-Cox) to stabilize skewed data. Where appropriate, unsupervised techniques (PCA/autoencoders) reduce dimensionality. High-cardinality features (e.g. employer name) use target-based encodings or embedding layers. As in credit modeling practice, each engineered feature is documented with monotonicity (“higher is better” or vice versa) to aid explanations.

Feature Store: Computed feature values are stored centrally so that the online scoring service fetches exactly the same values as used in training. This avoids “training-serving skew.” The store maintains both raw and normalized values; the API queries this store when scoring incoming requests.

Model Training & Explainability

Model Training: The training pipeline (orchestrated offline) fits the models on historical data. We use gradient-boosting models (LightGBM/CatBoost) due to their strong performance on tabular data. A GRU network is trained on any available temporal sequence (e.g. monthly payments) to detect late-payment patterns. We perform cross-validation and hyperparameter tuning for

accuracy. After initial training, we calibrate predicted probabilities (e.g. isotonic regression) so that scores correspond to true default rates.

Explainability: Following RBI's FREE-AI recommendations, we integrate explainability tools into the pipeline. For each prediction:

SHAP Values: Compute SHAP values for the chosen model to quantify each feature's positive or negative contribution. These are visualized as bar charts or waterfalls on the UI.

Reason Codes: We extract the top-N SHAP features as plain-language reasons (e.g. "Income: ₹30,000 vs typical ₹50,000; Credit card balance high"). This mirrors industry practice (Equifax's model produces personalized reason codes).

Counterfactuals: If an applicant is denied, we generate a counterfactual explanation: *"If your annual income were ₹X instead of ₹Y (and all else equal), you would have been approved"*. This is done with a technique like DiCE or custom search for minimal changes.

Global Interpretability: We also track global feature importance and partial dependence curves to ensure the model aligns with domain knowledge (e.g. monotonicity). We will manually inspect these during validation to detect any bias or unexpected behavior.

Regulatory Compliance: By design, all explanations are logged and linked to the model version, fulfilling requirements for an audit trail. The system can output model diagnostic reports on demand, as RBI suggests (using tools like SHAP or LIME) to demonstrate fairness and lack of bias.

Indian-Specific Design Considerations

This proof-of-concept is tailored for India's lending context:

Informal Economy: Over 90% of Indian workers are in informal employment. Income can be volatile and undocumented, so our model uses proxy signals (e.g. length of tenure, borrowing history). We allow users to enter income as a range or seasonal values, and create features capturing income stability.

Digital Access: Roughly half of India's mobile users still have feature phones. While our PoC is a web app, the UI is designed mobile-first with large inputs and local-language support. In future, the same scoring API could back voice- or SMS-based front ends (USSD/IVR) for non-smartphone users.

Regulatory Transparency: RBI's Digital Lending guidelines mandate that lenders explain key terms (interest, fees) and credit decisions to customers. Thus the UI will clearly display these elements alongside the score. The model itself is designed to minimize hidden biases: we follow RBI's "auditable algorithm" advice and avoid using sensitive attributes.

Inclusion: RBI requires that even borrowers with no formal credit history be gradually brought

into the system. Our prototype accommodates “thin-file” cases by relying more on alternative data (transactions) and providing provisional scores. We do not connect to real credit bureaus in the PoC, but our feature set mimics bureau features where possible (e.g. count of past loans).

Ethical AI: We heed RBI’s emphasis on fairness and ethics. During validation, we will check for disparate impact (e.g. no unfair denials for any community) and aim to mitigate biases by adjusting training weights or models. This aligns with RBI’s requirement for “fairness by design.”

Data Privacy: The design avoids unnecessary personal data. We comply with India’s DPDP Act (2023) and RBI’s call to “avoid over-collection”. All user data is encrypted at rest, and the service could be deployed on-premises at a lender if needed.

Deployment Format

The system is fully containerized for easy deployment. The backend API and explainability service run in one or more Docker containers, and the Next.js frontend is served from another container. For example, a Kubernetes cluster (or even a single Docker Compose setup) can host the containers, with environment variables controlling model paths. Model artifacts and feature-store snapshots are versioned (e.g. with MLflow or Git LFS) and mounted into the API container. A continuous integration pipeline can rebuild and redeploy the API whenever a new model is trained. This Docker-based approach ensures the PoC can be demonstrated on any cloud or on-prem machine with minimal setup.

Intended Use Cases for Indian Lenders

This prototype serves as an analytics and decision-support tool for lenders, particularly:

Digital Lenders/NBFCs: Fintech lenders can use it to pre-screen small personal or business loans where quick credit checks and clear explanations improve customer trust.

Commercial Banks: As a supplement to formal credit bureau scores for salaried and self-employed customers, especially in rural or semi-urban branches.

Microfinance & Co-operative Banks: Assessing underbanked borrowers (e.g. micro-entrepreneurs) by incorporating alternate data and transparent rationale.

Risk Analysts / Regulators: As a sandbox to study how AI models make loan decisions, to validate compliance with RBI’s “Explainable AI” expectations.
In all cases, the engine is not a final authority but a decision aid: it can flag high-risk applicants or justify loan pricing, while lending officers review the reasons codes and counterfactuals before acting.

Limitations of the PoC Design

Data Validity: Models are trained on Kaggle/benchmark data, which may not fully reflect current Indian borrower behavior. Performance on real Indian data may differ.

No Live Integrations: This prototype does not connect to actual credit bureaus, bank systems, or identity systems (Aadhaar, PAN). In practice, those integrations (and strict KYC/AML checks) are needed.

Scalability: The PoC is designed for demonstration. It lacks a production-grade environment (no cluster auto-scaling, limited fault-tolerance). High-volume throughput would need further engineering.

Simplified UI/UX: The frontend is minimal and suited for demonstration. A real product would require rigorous UX design for all user demographics (regional languages, low literacy, etc.).

Regulatory Validation: While we emphasize explainability, the model has not been vetted by RBI or any authority. It should not be used for actual lending without thorough third-party validation and compliance checks.

Model Risk: Like any ML model, ours can degrade over time (model drift) or exhibit unintended biases. Ongoing monitoring and human oversight are required.

In summary, this project outlines a transparent, explainable credit risk engine prototype for the Indian market. It combines a modern ML stack (GBMs, GRUs) with an emphasis on regulatory compliance (SHAP explanations, reason codes, counterfactuals) and addresses local constraints (informal incomes, feature-phone UI). While not production-ready, it provides a concrete framework for lenders to experiment with AI-driven credit decisions under India's evolving regulatory landscape (e.g. RBI's FREE-AI guidelines).

Sources: RBI and World Bank guidelines on explainable credit models; Indian fintech UX design (feature-phone usage); India labor statistics (informal employment); RBI digital lending rules (transparency, bureau reporting); XAI methods in credit (SHAP, reason codes, counterfactuals); feature engineering best practices; and modern underwriting architecture.